

Potencia estadística

Jake Bowers — EGAP Learning Days, Santiago de Chile

2026-07-27

Tabla de contenidos I

- 1 ¿Qué es la potencia estadística?
- 2 ¿Qué aumenta o reduce la potencia?
- 3 Cálculos analíticos de potencia
- 4 Cálculo de la potencia basado en simulación
- 5 Potencia con ajuste de covariables
- 6 Potencia para la aleatorización por conglomerados

Sección 1

¿Qué es la potencia estadística?

¿Qué es la potencia estadística?

Queremos separar la señal del ruido.

- Potencia (o “poder estadístico”) = probabilidad de rechazar la hipótesis nula, dado un efecto verdadero $\neq 0$. Queremos que nuestras pruebas produzcan $p \leq \alpha$ para la hipótesis de ningún efecto *cuando la verdad no es cero*.
- Es la capacidad de una prueba de detectar un efecto, dado que el efecto existe.
- Formalmente: $1 -$ (tasa de error de tipo II) de una prueba de hipótesis.
- Por lo tanto, la potencia está $\in (0, 1)$.
- ¿Con qué frecuencia deberíamos ver un $p \leq \alpha$? Umbrales estándar: 0.8 o 0.9 — “detectar la señal casi siempre que exista”.
- La “potencia” es una característica operativa de las pruebas de hipótesis (y, por extensión, de los intervalos de confianza). Los estimadores no tienen potencia estadística.

Punto de partida para el análisis de potencia

- El análisis de potencia es algo que hacemos *antes* de realizar un estudio.
 - Nos ayuda a determinar **cuántas unidades** necesitamos para detectar un efecto de un tamaño dado.
 - Nos ayuda a determinar la **diferencia mínima detectable** dado un tamaño de muestra fijo.
 - **Puede ayudarnos a decidir si vale la pena realizar el estudio.** (El análisis de potencia es parte de responder la pregunta: “¿Deberíamos hacer este estudio?”)
- Meta: descubrir si el diseño que planeamos nos permite detectar los efectos que esperamos (o los efectos que necesitaríamos ver antes de actuar) en caso de que existan.
- Es difícil aprender de un hallazgo nulo con poca potencia.
 - ¿Había un efecto que no pudimos detectar? ¿O no había efecto? No podemos distinguir.

Potencia

- Supongamos que el tratamiento sí tiene un efecto y que repetimos el experimento muchas veces. ¿Con qué frecuencia obtendríamos $p \leq .05$?
- Depende:
 - ¿Qué tan grande es el efecto del tratamiento?
 - ¿Cuántas unidades reciben el tratamiento? ¿Cuántas se miden?
 - ¿Cuánto ruido hay en la medición de la variable de resultado?
- **No conocemos de antemano las respuestas a todas estas preguntas. Así que responderlas requiere hacer conjeturas.**

Análisis de potencia

El análisis de potencia nos muestra cuánto cambia la probabilidad de obtener un resultado significativo cuando cambiamos los supuestos.

- Muchas disciplinas han adoptado una potencia objetivo de 0.80. ¿Razones para subir a 0.90?
- Por ejemplo: si con nuestros supuestos actuales la potencia es 0.70, ¿qué tendríamos que cambiar para llegar a 0.80? ¿Más unidades? ¿Una variable de resultado con menos ruido? ¿Un tratamiento más fuerte?
- Podemos ajustar el diseño y los supuestos hasta confiar en que el experimento producirá resultados estadísticamente significativos “la mayoría” de las veces (donde “la mayoría” = 80 % o 90 %).
- Requiere muchas conjeturas informadas, pero suele moderar nuestras expectativas: si la literatura previa siempre ha encontrado un tamaño de efecto de 1 desviación estándar, puede que no tengamos el presupuesto para detectar un efecto de ese tamaño.

Sección 2

¿Qué aumenta o reduce la potencia?

La potencia crece con N !

```
## Con sd = 1, delta queda en unidades de desviación estándar de la variable
## de resultado: delta = 0.25 es un efecto de 0.25 desviaciones estándar. Ese
## efecto estandarizado es la d de Cohen: d = (m_1 - m_2) / sigma.
```

```
power.t.test(n = 100, delta = .25, sd = 1, sig.level = .05)
```

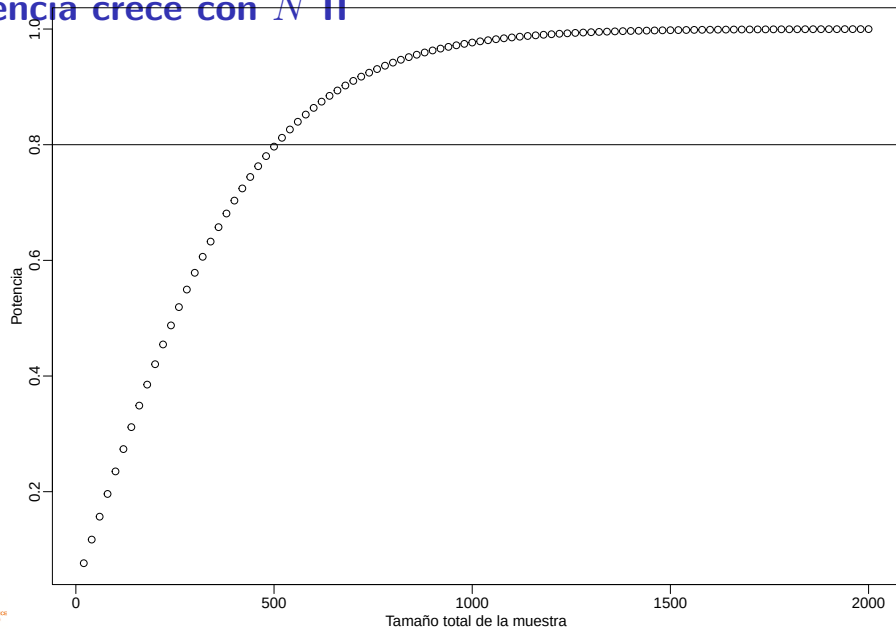
Two-sample t test power calculation

```
      n = 100
  delta = 0.25
      sd = 1
sig.level = 0.05
  power = 0.4204
alternative = two.sided
```

NOTE: n is number in *each* group

```
some_ns <- seq(10, 1000, by = 10)
pow_by_n <- sapply(some_ns, function(n) {
  power.t.test(n = n, delta = 0.25, sd = 1, sig.level = 0.05)$power
  # pwr.t.test(n = n, d = 0.25, sig.level = 0.05)$power
})
```

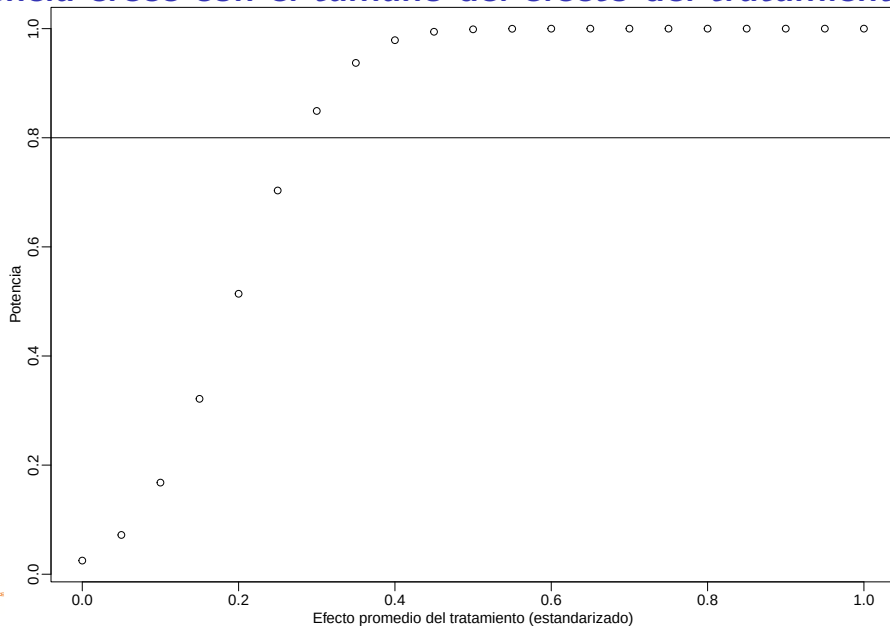
La potencia crece con N II



Estrategias para aumentar el número de unidades

- Aumentar el presupuesto
- Ampliar la definición de la población (p. ej., de ciudad a región, extender el tiempo)
- Reducir el tamaño de los conglomerados (p. ej., de escuelas a salones de clase)

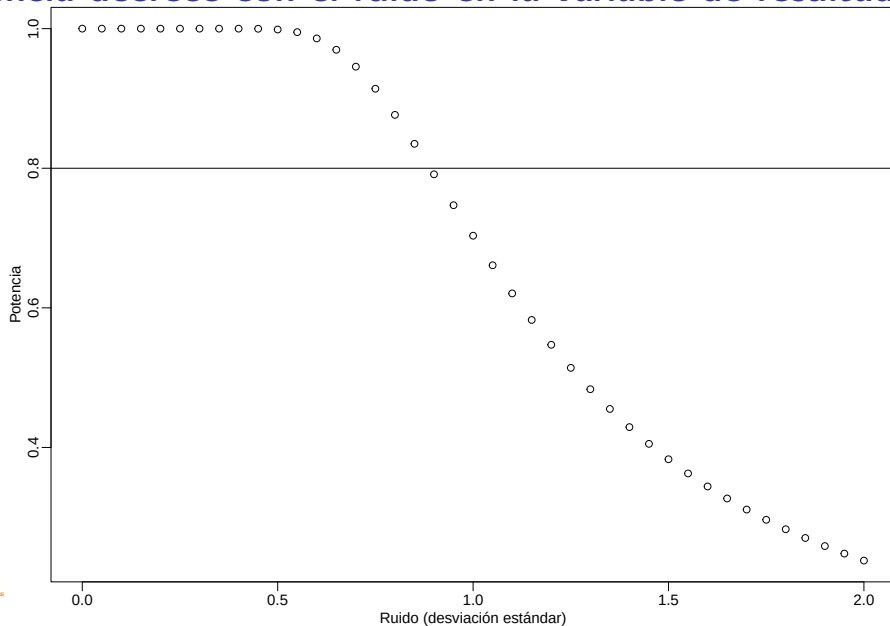
La potencia crece con el tamaño del efecto del tratamiento



Estrategias para aumentar el efecto del tratamiento

- ¿Qué efectos mínimos querrían poder demostrar con su estudio?
 - ¿Qué efecto es realista?
 - En trabajo colaborativo: ¿qué efecto es deseable?
 - ¿Sería “satisfactorio” este tamaño de efecto? (costo-efectividad, otros estudios, reglas generales)
- Aumentar la dosis / evitar tratamientos muy débiles
 - ¿Uno o varios brazos de tratamiento?
 - ¿Efecto del tratamiento sobre qué?
 - Evitar variables de resultado cerca del piso o del techo
 - ¿Cuándo medir la variable de resultado?

La potencia decrece con el ruido en la variable de resultado



El ruido de la variable de resultado (σ)

- Reducir el ruido. ¿Cómo?
 - La medición, el intervalo de tiempo
 - Bloques. Realizar el experimento entre sujetos más parecidos entre sí
 - Recolectar covariables de línea de base y usar los residuos de la variable de resultado, o el ajuste directo de covarianza (en experimentos suficientemente grandes)
 - Recolectar varias medidas de los resultados y (1) probar hipótesis combinadas ("ningún efecto sobre ninguna de Y_1, Y_2, Y_3 ") o (2) crear un índice.

Métodos para calcular la potencia

- Cálculos analíticos de potencia
- Simulación

Herramientas para el cálculo de la potencia

- Interactivas
 - Calculadora de potencia de EGAP
 - [rpsychologist](#)
- Paquetes de R
 - `power.t.test` (incluida en R base)
 - [pwr](#)
 - [DeclareDesign](#), ver también <https://declaredesign.org/>

Sección 3

Cálculos analíticos de potencia

Cálculos analíticos de potencia para hipótesis de ningún efecto promedio del tratamiento

Las gráficas anteriores provienen de esta fórmula.

- Fórmula:

$$\text{Potencia} = \Phi \left(\frac{|\tau|\sqrt{N}}{2\sigma} - \Phi^{-1}\left(1 - \frac{\alpha}{2}\right) \right)$$

- Componentes:

- ϕ : la función de distribución acumulada (CDF) de la normal estándar, que crece monotónicamente
- τ : el tamaño del efecto
- N : el tamaño de la muestra
- σ : la desviación estándar de la variable de resultado
- α : el nivel de significancia (normalmente 0.05)

¿Por qué la normal estándar? (por el Teorema del Límite Central (TLC))

Ejemplo: Cálculos analíticos de potencia

```
# Potencia para un estudio con 80 observaciones, un tamaño de
# efecto de 0.25 y una desviación estándar de 1
power.t.test(
  n = 40, delta = 0.25, sd = 1,
  sig.level = 0.05, power = NULL
)
```

Two-sample t test power calculation

```
      n = 40
  delta = 0.25
     sd = 1
sig.level = 0.05
  power = 0.1961
alternative = two.sided
```

NOTE: n is number in *each* group

Ejemplo: Cálculos analíticos de potencia

```
power.t.test(  
  n = 40, delta = NULL, sd = 0.5,  
  sig.level = 0.05, power = 0.8  
)
```

Two-sample t test power calculation

```
      n = 40  
delta = 0.3171  
      sd = 0.5  
sig.level = 0.05  
      power = 0.8  
alternative = two.sided
```

NOTE: n is number in *each* group

Ejemplo: Cálculos analíticos de potencia

```
power.t.test(  
  n = NULL, delta = 0.25, sd = 1,  
  sig.level = 0.05, power = 0.8  
)
```

Two-sample t test power calculation

```
      n = 252.1  
delta = 0.25  
  sd = 1  
sig.level = 0.05  
  power = 0.8  
alternative = two.sided
```

NOTE: n is number in *each* group

Limitaciones de los cálculos analíticos de potencia

- Solo se han derivado para algunas estadísticas de prueba (diferencias de medias)
- Implican supuestos específicos acerca del proceso de generación de los datos
- Son incompatibles con diseños más complejos, como los diseños aleatorizados por bloques con distintas probabilidades de asignación en cada bloque.

Sección 4

Cálculo de la potencia basado en simulación

Pasos para el cálculo de la potencia basado en simulación

En general:

- Crear bases de datos a partir de la literatura previa y del conocimiento del contexto.
- Simular el diseño de investigación y el análisis muchas veces.
- ¿Cuántas veces obtuvieron $p \leq .05$ cuando fijaron $\tau = .25$? Esa es la potencia para detectar un efecto de $\tau = .25$.

Los estudios de simulación también requieren supuestos, pero ustedes definen los suyos.

Cálculo de la potencia basado en simulación con DeclareDesign

- Miembros de EGAP crearon un paquete de R llamado DeclareDesign (y escribieron un libro sobre simulaciones para diseños de investigación); ver <https://declareddesign.org/> Blair et al. (2023)
- Ayuda a evaluar nuestros diseños con múltiples “diagnosands” (criterios de diagnóstico)
 - Potencia (de las pruebas)
 - Sesgo (de los estimadores)
 - RMSE (de los estimadores)
 - Cobertura (de los intervalos de confianza)

Pasos en DeclareDesign

- Definir la muestra y la función de resultados potenciales.
- Definir el procedimiento de asignación del tratamiento.
- Crear los datos.
- Asignar el tratamiento y luego probar una hipótesis (usando un valor p o un intervalo de confianza)
- Repetir esto muchas veces.

Ejemplos

- Aleatorización completa
- Agregando una covariable
- Aleatorización por conglomerados

Ejemplo: Potencia basada en simulación para la aleatorización completa

Primero, usando código R básico:

```
# install.packages("randomizr")
library(randomizr)
library(estimatr)

make_Y0 <- function(N) {
  rnorm(n = N)
}

repeat_experiment_and_test <- function(N, Y0, tau) {
  # N es el tamaño del grupo experimental; Y0 es el resultado potencial bajo control
  # tau es el tamaño del efecto (aquí, un efecto aditivo constante)
  Y1 <- Y0 + tau
  Z <- complete_ra(N = N)
  Yobs <- Z * Y1 + (1 - Z) * Y0
  estimator <- lm_robust(Yobs ~ Z)
  pval <- estimator$p.value[2]
  return(pval)
}
```

Ejemplo: Potencia basada en simulación para la aleatorización completa

```
power_sim <- function(N, tau, sims) {  
  ## Y0 es fijo en la mayoría de los experimentos de campo.  
  ## Así que lo generamos solo una vez:  
  Y0 <- make_Y0(N)  
  ## Y luego repetimos la asignación experimental y el análisis:  
  pvals <- replicate(  
    n = sims,  
    repeat_experiment_and_test(N = N, Y0 = Y0, tau = tau)  
  )  
  pow <- sum(pvals < .05) / sims  
  return(pow)  
}  
  
set.seed(12345)  
## Noten la variabilidad de simulación en simulación con sims=100  
power_sim(N = 80, tau = .25, sims = 100)
```

```
[1] 0.15
```

```
power_sim(N = 80, tau = .25, sims = 100)
```

```
[1] 0.21
```

Ejemplo: Usando DeclareDesign I

```
library(DeclareDesign)
library(tidyverse)
## DeclareDesign 1.1+ requiere que parámetros como N y tau existan cuando se
## declara el diseño; redesign() abajo los reemplaza con otros valores.
N <- 100
tau <- .25
P0 <- declare_population(N, u0 = rnorm(N))
# declarar Y(Z=1) y Y(Z=0), donde Y(Z=0) tiene media 5
O0 <- declare_potential_outcomes(Y_Z_0 = 5 + u0, Y_Z_1 = Y_Z_0 + tau)
# Asignar m unidades al tratamiento
A0 <- declare_assignment(Z = conduct_ra(N = N, m = round(N / 2)))
# el estimando es la diferencia promedio entre Y(Z=1) y Y(Z=0)
estimand_ate <- declare_inquiry(ATE = mean(Y_Z_1 - Y_Z_0))
R0 <- declare_reveal(Y, Z)
design0_base <- P0 + A0 + O0 + R0

## Por ejemplo, con N=100 y tau=.25:
design0_N100_tau25 <- redesign(design0_base, N = 100, tau = .25)
dat0_N100_tau25 <- draw_data(design0_N100_tau25)
head(dat0_N100_tau25)
```

Ejemplo: Usando DeclareDesign II

	ID	u0	Z	Y_Z_0	Y_Z_1	Y
1	001	-0.2060	0	4.794	5.044	4.794
2	002	-0.5875	0	4.413	4.663	4.413
3	003	-0.2908	1	4.709	4.959	4.959
4	004	-2.5649	0	2.435	2.685	2.435
5	005	-1.8967	0	3.103	3.353	3.103
6	006	-1.6401	1	3.360	3.610	3.610

```
# ATE verdadero
```

```
with(dat0_N100_tau25, mean(Y_Z_1 - Y_Z_0))
```

```
[1] 0.25
```

```
# ATE estimado
```

```
with(dat0_N100_tau25, mean(Y[Z == 1]) - mean(Y[Z == 0]))
```

```
[1] 0.5569
```

```
# Usando el estimador del ATE como estadística de prueba para probar la
```

```
# hipótesis nula de ningún efecto promedio:
```

```
lm_robust(Y ~ Z, data = dat0_N100_tau25)$p.value[2]
```

```
Z  
0.002609
```


Ejemplo: Usando DeclareDesign III

```
E0 <- declare_estimator(Y ~ Z,
  .method = lm_robust, label = "t test 1",
  inquiry = "ATE"
)

t_test <- function(data) {
  # test <- with(data, t.test(x = Y[Z == 1], y = Y[Z == 0]))
  test <- coin::oneway_test(Y ~ factor(Z), data = data, distribution = "asymptotic")
  data.frame(statistic = coin::statistic(test), p.value = coin::pvalue(test))
}

T0 <- declare_test(handler = label_test(t_test), label = "t test 2")
design0_plus_tests <- design0_base + E0 + T0

design0_N100_tau25_plus <- redesign(design0_plus_tests, N = 100, tau = .25)

## Solo repetimos la asignación aleatoria, no la creación de Y0.
## Ignoren la advertencia
names(design0_N100_tau25_plus)

[1] "P0"      "A0"      "00"      "R0"      "t test 1" "t test 2"
str(design0_N100_tau25_plus)
```

Ejemplo: Usando DeclareDesign IV

List of 6

```
$ P0      :design_step: declare_population(N, u0 = rnorm(N))
$ A0      :design_step: declare_assignment(Z = conduct_ra(N = N, m = round(N/2)))
$ O0      :design_step: declare_potential_outcomes(Y_Z_0 = 5 + u0, Y_Z_1 = Y_Z_0 + tau)
$ R0      :design_step: declare_reveal(Y, Z)
$ t test 1:design_step: declare_estimator(Y ~ Z, .method = lm_robust, inquiry = "ATE", label = "t test 1")
$ t test 2:design_step: declare_test(handler = label_test(t_test), label = "t test 2")
- attr(*, "call")= language construct_design(steps = steps)
- attr(*, "class")= chr [1:2] "design" "dd"
- attr(*, "parameters")=List of 2
..$ N : chr "100"
..$ tau: chr "0.25"
```

```
design0_N100_tau25_sims <- simulate_design(design0_N100_tau25_plus,
  sims = c(1, 100, 1, 1, 1, 1)
) # solo repetir la asignación aleatoria
# design0_N100_tau25_sims tiene 200 filas (2 pruebas * 100 asignaciones aleatorias)
# veamos solo algunos de los valores
str(design0_N100_tau25_sims)
```

Ejemplo: Usando DeclareDesign V

```
'data.frame': 200 obs. of 17 variables:
 $ design      : chr  "design0_N100_tau25_plus" "design0_N100_tau25_plus" "design0_N100_tau25_plus" "design0_N100_tau25_p
 $ N           : int  100 100 100 100 100 100 100 100 100 100 ...
 $ tau         : num  0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 ...
 $ sim_ID      : int  1 1 2 2 3 3 4 4 5 5 ...
 $ estimator   : chr  "t test 1" "t test 2" "t test 1" "t test 2" ...
 $ term        : chr  "Z" NA "Z" NA ...
 $ estimate    : num  0.111 NA 0.246 NA 0.546 ...
 $ std.error    : num  0.215 NA 0.215 NA 0.213 ...
 $ statistic    : num  0.515 -0.517 1.141 -1.139 2.561 ...
 $ p.value     : num  0.608 0.605 0.257 0.255 0.012 ...
 $ conf.low     : num  -0.316 NA -0.182 NA 0.123 ...
 $ conf.high    : num  0.537 NA 0.673 NA 0.97 ...
 $ df          : num  98 NA 98 NA 98 NA 98 NA 98 NA ...
 $ outcome     : chr  "Y" NA "Y" NA ...
 $ inquiry     : chr  "ATE" NA "ATE" NA ...
 $ step_1_draw : num  1 1 1 1 1 1 1 1 1 1 ...
 $ step_2_draw : num  1 1 2 2 3 3 4 4 5 5 ...
- attr(*, "parameters")='data.frame': 1 obs. of 3 variables:
 ..$ design: chr "design0_N100_tau25_plus"
 ..$ N      : int 100
 ..$ tau    : num 0.25
```

Potencia con aleatorización completa

En el 20 % de los experimentos, cuando el efecto verdadero es de 0.25 desviaciones estándar y $N = 100$, obtenemos $p < .05$.

```
# para cada estimador, potencia = proporción de simulaciones con p.value < 0.05
design0_N100_tau25_sims %>%
  group_by(estimator) %>%
  summarize(pow = mean(p.value < .05), mean_p = mean(p.value), .groups = "drop")
```

```
# A tibble: 2 x 3
  estimator    pow mean_p
  <chr>      <dbl> <dbl>
1 t test 1    0.2  0.338
2 t test 2    0.2  0.337
```

Sección 5

Potencia con ajuste de covariables

Ajuste de covariables y potencia

- El ajuste de covariables/covarianza puede mejorar la potencia porque reduce la variación de la variable de resultado.
 - Si las covariables son pronósticas (predicen la variable de resultado), el ajuste puede reducir la varianza drásticamente. Menor varianza significa mayor potencia.
 - Si no son pronósticas, las ganancias en potencia son mínimas.
- Todas las covariables deben ser pre-tratamiento. No eliminan observaciones por datos faltantes.
 - Ver el módulo de [amenazas a la validez interna](#) y [10 cosas que debe saber sobre el ajuste de covariables](#).
 - Freedman (2008) señaló que los estimadores del ATE ajustados por covarianza son sesgados. Lin (2013) muestra que el sesgo decrece con N y ofrece métodos que reducen el sesgo.

Bloques

- Bloques: asignar el tratamiento al azar dentro de los bloques
 - Ajuste de covariables “ex ante”
 - Mayor precisión/eficiencia implica más potencia
 - Reducir el “sesgo condicional”: la asociación entre la asignación del tratamiento y los resultados potenciales
 - Los beneficios de los bloques frente al ajuste de covariables son más claros en experimentos pequeños

Ejemplo: Potencia basada en simulación con una covariable

```
## Y0 es fijo en la mayoría de los experimentos de campo.  
## Así que lo generamos solo una vez  
make_Y0_cov <- function(N) {  
  u0 <- rnorm(n = N)  
  x <- rpois(n = N, lambda = 2)  
  Y0 <- .5 * sd(u0) * x + u0  
  return(data.frame(Y0 = Y0, x = x))  
}  
## X predice Y0 moderadamente.  
test_dat <- make_Y0_cov(100)  
test_lm <- lm_robust(Y0 ~ x, data = test_dat)  
summary(test_lm)
```

Call:

```
lm_robust(formula = Y0 ~ x, data = test_dat)
```

Standard error type: HC2

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	CI Lower	CI Upper	DF
(Intercept)	0.11	0.1880	0.585	0.559753653	-0.263	0.483	98
x	0.44	0.0814	5.413	0.000000441	0.279	0.602	98

Multiple R-squared: 0.231 , Adjusted R-squared: 0.223

F-statistic: 29.3 on 1 and 98 DF, p-value: 0.000000441

Ejemplo: Potencia basada en simulación con una covariable I

Aquí lo hacemos a mano en vez de con DeclareDesign (ambas opciones son posibles).

```
## ahora preparamos la simulación
repeat_experiment_and_test_cov <- function(N, tau, Y0, x) {
  Y1 <- Y0 + tau
  Z <- complete_ra(N = N)
  Yobs <- Z * Y1 + (1 - Z) * Y0
  estimator <- lm_robust(Yobs ~ Z + x, data = data.frame(Y0, Z, x))
  pval <- estimator$p.value[2]
  return(pval)
}

## crear los datos una vez, asignar aleatoriamente el tratamiento sims veces
## reportar qué proporción produce un valor  $p < 0.05$ 
power_sim_cov <- function(N, tau, sims) {
  dat <- make_Y0_cov(N)
  pvals <- replicate(n = sims, repeat_experiment_and_test_cov(
    N = N,
    tau = tau, Y0 = dat$Y0, x = dat$x
  ))
  pow <- sum(pvals < .05) / sims
  return(pow)
}
```

Ejemplo: Potencia basada en simulación con una covariable I

Lo hacemos dos veces para dejar claro que hay variabilidad de simulación en simulación.

```
set.seed(12345)  
power_sim_cov(N = 80, tau = .25, sims = 100)
```

```
[1] 0.13
```

```
power_sim_cov(N = 80, tau = .25, sims = 100)
```

```
[1] 0.19
```

Sección 6

Potencia para la aleatorización por conglomerados

Potencia y diseños por conglomerados

- Dado un N fijo, un diseño por conglomerados tiene (débilmente) menos potencia que un diseño sin conglomerados.
 - La diferencia suele ser sustancial.
- Tenemos que estimar la varianza tomando en cuenta la dependencia:
 - Errores estándar agrupados por conglomerado (lo habitual)
 - Inferencia basada en la aleatorización (randomization inference)
- Para aumentar la potencia:
 - Es mejor aumentar el número de conglomerados que el número de unidades por conglomerado.
 - Cuánto reducen la potencia los conglomerados depende fundamentalmente de la correlación intra-clase (la razón entre la varianza dentro de los conglomerados y la varianza total).

Ejemplo: Potencia basada en simulación para la aleatorización por conglomerados I

Aquí fijamos ICC=.1

```
## Y0 es fijo en la mayoría de los experimentos de campo.  
## Así que lo generamos solo una vez  
make_Y0_clus <- function(n_indivs, n_clus) {  
  # n_indivs es el número de personas por conglomerado  
  # n_clus es el número de conglomerados  
  clus_id <- gl(n_clus, n_indivs)  
  N <- n_clus * n_indivs  
  u0 <- fabricatr::draw_normal_icc(N = N, clusters = clus_id, ICC = .1)  
  Y0 <- u0  
  return(data.frame(Y0 = Y0, clus_id = clus_id))  
}  
  
test_dat <- make_Y0_clus(n_indivs = 10, n_clus = 100)  
# confirmar que esto produce datos con 10 personas en cada uno de los 100 conglomerados  
table(test_dat$clus_id)
```

Ejemplo: Potencia basada en simulación para la aleatorización por conglomerados II

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10
34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10
67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96
10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10
100																													
10																													

```
# confirmar el ICC  
ICC::ICCbare(y = Y0, x = clus_id, data = test_dat)
```

```
[1] 0.09655
```

Ejemplo: Potencia basada en simulación para la aleatorización por conglomerados

```
repeat_experiment_and_test_clus <- function(N, tau, Y0, clus_id) {  
  Y1 <- Y0 + tau  
  # aquí aleatorizamos Z al nivel del conglomerado  
  Z <- cluster_ra(clusters = clus_id)  
  Yobs <- Z * Y1 + (1 - Z) * Y0  
  estimator <- lm_robust(Yobs ~ Z,  
    clusters = clus_id,  
    data = data.frame(Y0, Z, clus_id), se_type = "CR2"  
  )  
  pval <- estimator$p.value[2]  
  return(pval)  
}  
  
power_sim_clus <- function(n_indivs, n_clus, tau, sims) {  
  dat <- make_Y0_clus(n_indivs, n_clus)  
  N <- n_indivs * n_clus  
  # aleatorizar el tratamiento sims veces  
  pvals <- replicate(  
    n = sims,  
    repeat_experiment_and_test_clus(  
      N = N, tau = tau,  
      Y0 = dat$Y0, clus_id = dat$clus_id  
    )  
  )  
  pow <- sum(pvals < .05) / sims  
  return(pow)  
}
```

Ejemplo: Potencia basada en simulación para la aleatorización por conglomerados I

```
set.seed(12345)  
power_sim_clus(n_indivs = 8, n_clus = 100, tau = .25, sims = 100)
```

```
[1] 0.66
```

```
power_sim_clus(n_indivs = 8, n_clus = 100, tau = .25, sims = 100)
```

```
[1] 0.68
```


Ejemplo: Potencia basada en simulación para la aleatorización por conglomerados (DeclareDesign)

```
## Valores iniciales para que las declaraciones se construyan;  
## redesign() los varía abajo  
n_clus <- 10  
n_indivs <- 100  
P1 <- declare_population(  
  N = n_clus * n_indivs,  
  clusters = gl(n_clus, n_indivs),  
  u0 = draw_normal_icc(N = N, clusters = clusters, ICC = .2)  
)  
O1 <- declare_potential_outcomes(Y_Z_0 = 5 + u0, Y_Z_1 = Y_Z_0 + tau)  
A1 <- declare_assignment(Z = conduct_ra(N = N, clusters = clusters))  
estimand_ate <- declare_inquiry(ATE = mean(Y_Z_1 - Y_Z_0))  
R1 <- declare_reveal(Y, Z)  
design1_base <- P1 + A1 + O1 + R1 + estimand_ate
```

Ejemplo: Potencia basada en simulación para la aleatorización por conglomerados (DeclareDesign)

```
## Por ejemplo:
design1_test <- redesign(design1_base, n_clus = 10, n_indivs = 100, tau = .25)
test_d1 <- draw_data(design1_test)
# confirmar que todos los individuos de un conglomerado tienen la misma
# asignación de tratamiento
with(test_d1, table(Z, clusters))

      clusters
Z      1  2  3  4  5  6  7  8  9 10
0 100  0 100 100 100  0  0 100  0  0
1  0 100  0  0  0 100 100  0 100 100

# tres estimadores, difieren en se_type:
E1a <- declare_estimator(Y ~ Z,
  .method = lm_robust, clusters = clusters,
  se_type = "CR2", label = "CR2 cluster t test",
  inquiry = "ATE"
)
E1b <- declare_estimator(Y ~ Z,
  .method = lm_robust, clusters = clusters,
  se_type = "CRO", label = "CRO cluster t test",
  inquiry = "ATE"
)
E1c <- declare_estimator(Y ~ Z,
  .method = lm_robust, clusters = clusters,
  se_type = "stata", label = "stata RCSE t test",
  inquiry = "ATE"
)
E1d <- declare_estimator(Y ~ Z,
  .method = lm_robust,
  se_type = "classical", label = "plain IID OLS t test",
  inquiry = "ATE"
)

design1_plus <- design1_base + E1a + E1b + E1c + E1d

design1_plus_tosim <- redesign(design1_plus, n_clus = 10, n_indivs = 100, tau = .25)
```

Ejemplo: Potencia basada en simulación para la aleatorización por conglomerados (DeclareDesign)

```
## Solo repetir la asignación aleatoria, no la creación de Y0.  
## Ignoren la advertencia  
## En la práctica querríamos más simulaciones.  
set.seed(12355)  
design1_sims <- simulate_design(design1_plus_tosim,  
  sims = c(1, 1000, rep(1, length(design1_plus_tosim) - 2))  
)
```

Potencia basada en simulación para la aleatorización por conglomerados

Noten: mucha potencia pero poca cobertura para el OLS simple (la “cobertura” de un intervalo de confianza es el complemento de la tasa de falsos positivos de una prueba de hipótesis: 95 % de cobertura corresponde a una tasa de falsos positivos del 5 %).

```
design1_sims %>%
  group_by(estimator) %>%
  summarize(
    pow = mean(p.value < .05),
    coverage = mean(estimand <= conf.high & estimand >= conf.low),
    .groups = "drop"
  )
```

A tibble: 4 x 3

estimator	pow	coverage
<chr>	<dbl>	<dbl>
1 CR0 cluster t test	0.155	0.911
2 CR2 cluster t test	0.105	0.936
3 plain IID OLS t test	0.723	0.333
4 stata RCSE t test	0.131	0.918

Ejemplo: Potencia basada en simulación para la aleatorización por conglomerados (DeclareDesign)

```
library(DesignLibrary)
## Esto puede ser más simple que lo anterior:
d1 <- block_cluster_two_arm_designer(
  N_blocks = 1,
  N_clusters_in_block = 10,
  N_i_in_cluster = 100,
  sd_block = 0,
  sd_cluster = .3,
  ate = .25
)
d1_plus <- d1 + E1b + E1c
d1_sims <- simulate_design(d1_plus, sims = c(1, 1, 1000, 1, 1, 1, 1, 1))
```

Ejemplo: Potencia basada en simulación para la aleatorización por conglomerados (DeclareDesign)

```
d1_sims %>%  
  group_by(estimator) %>%  
  summarize(  
    pow = mean(p.value < .05),  
    coverage = mean(estimand <= conf.high & estimand >= conf.low),  
    .groups = "drop"  
  )
```

A tibble: 3 x 3

estimator	pow	coverage
<chr>	<dbl>	<dbl>
1 CR0 cluster t test	0.209	0.914
2 estimator	0.143	0.941
3 stata RCSE t test	0.194	0.925

Los ingredientes de la potencia y su relación

- La potencia:
 - Crece con N (el número de unidades en el experimento)
 - Crece con $|\tau|$ (el tamaño del efecto del tratamiento)
 - Decrece con σ (la varianza de la variable de resultado)
 - Decrece con el ICC (la homogeneidad relativa de la variable de resultado dentro de los conglomerados frente a entre conglomerados)

Comentarios

- Conozcan bien su variable de resultado.
- ¿Qué efectos pueden esperar de manera realista de su tratamiento? (¿Estudios previos parecidos al suyo? ¿Dado lo que saben sobre la “variación natural” de la variable de resultado?)
- ¿Qué efectos son sustantivamente demasiado pequeños? (Puede que no valga la pena realizar *este experimento en este momento* si no pueden imaginar detectar un efecto al menos así de grande.)
- ¿Cuál es el rango plausible de variación de la variable de resultado?
- Un diseño con poco movimiento posible en la variable de resultado puede tener poca potencia.
 - Ver [10 cosas que su resultado nulo podría significar](#) para una discusión de las varias razones por las que un experimento dado pudo producir un $p > .05$ (si usan $\alpha = .05$ como criterio de rechazo, o si tienen algún $\hat{\tau}$ sustantivamente pequeño que quizás no se alcance).

Nota: los cálculos de potencia (resúmenes de la tasa de falsos negativos de las pruebas/intervalos de confianza) dependen de que esas pruebas/intervalos de confianza tengan tasas de falsos positivos controladas y bajas (p. ej., 5 % o menos) o tasas de cobertura altas (p. ej., 95 % o más)

Referencias

Blair, Graeme, Alexander Coppock, y Macartan Humphreys. 2023. *Research design in the social sciences: declaration, diagnosis, and redesign*. Princeton University Press.

Freedman, David A. 2008. «On regression adjustments to experimental data». *Advances in Applied Mathematics* 40 (2): 180-93.

Lin, Winston. 2013. «Agnostic notes on regression adjustments to experimental data: Reexamining Freedman's critique». *The Annals of Applied Statistics* 7 (1): 295-318.